15.2.9 LOAD DATA Statement

```
LOAD DATA
    [LOW_PRIORITY | CONCURRENT] [LOCAL]
    INFILE 'file_name'
    [REPLACE | IGNORE]
    INTO TABLE tbl_name
    [PARTITION (partition_name [, partition_name] ...)]
    [CHARACTER SET charset_name]
    [{FIELDS | COLUMNS}
        [TERMINATED BY 'string']
        [[OPTIONALLY] ENCLOSED BY 'char']
        [ESCAPED BY 'char']
    ]
    [LINES
        [STARTING BY 'string']
        [TERMINATED BY 'string']
    ]
    [IGNORE number {LINES | ROWS}]
    [(col_name_or_user_var
        [, col_name_or_user_var] ...)]
    [SET col_name={expr | DEFAULT}
        [, col_name={expr | DEFAULT}] ...]
```

The `LOAD DATA` statement reads rows from a text file into a table at a very high speed. The file can be read from the server host or the client host, depending on whether the `LOCAL` modifier is given. `LOCAL` also affects data interpretation and error handling.

`LOAD DATA` is the complement of `SELECT ... INTO OUTFILE`. (See Section 15.2.13.1, “SELECT ... INTO Statement”.) To write data from a table to a file, use `SELECT ... INTO OUTFILE`. To read the file back into a table, use `LOAD DATA`. The syntax of the `FIELDS` and `LINES` clauses is the same for both statements.

The **mysqlimport** utility provides another way to load data files; it operates by sending a `LOAD DATA` statement to the server. See Section 6.5.5, “mysqlimport — A Data Import Program”.

For information about the efficiency of `INSERT` versus `LOAD DATA` and speeding up `LOAD DATA`, see Section 10.2.5.1, “Optimizing INSERT Statements”.

- Non-LOCAL Versus LOCAL Operation
- Input File Character Set

- Input File Location
- Security Requirements
- Duplicate-Key and Error Handling
- Index Handling
- Field and Line Handling
- Column List Specification
- Input Preprocessing
- Column Value Assignment
- Partitioned Table Support
- Concurrency Considerations
- Statement Result Information
- Replication Considerations
- Miscellaneous Topics

Non-LOCAL Versus LOCAL Operation

The `LOCAL` modifier affects these aspects of `LOAD DATA`, compared to non-`LOCAL` operation:

- It changes the expected location of the input file; see Input File Location.
- It changes the statement security requirements; see Security Requirements.
- Unless `REPLACE` is also specified, `LOCAL` has the same effect as the `IGNORE` modifier on the interpretation of input file contents and error handling; see Duplicate-Key and Error Handling, and Column Value Assignment.

`LOCAL` works only if the server and your client both have been configured to permit it. For example, if **mysqld** was started with the `local_infile` system variable disabled, `LOCAL` produces an error. See Section 8.1.6, “Security Considerations for LOAD DATA LOCAL”.

Input File Character Set

The file name must be given as a literal string. On Windows, specify backslashes in path names as forward slashes or doubled backslashes. The server interprets the file name using the character set indicated by the `character_set_filesystem` system variable.

By default, the server interprets the file contents using the character set indicated by the `character_set_database` system variable. If the file contents use a character set different from this default, it is a good idea to specify that character set by using the `CHARACTER SET` clause. A character set of `binary` specifies "no conversion."

`SET NAMES` and the setting of `character_set_client` do not affect interpretation of file contents.

`LOAD DATA` interprets all fields in the file as having the same character set, regardless of the data types of the columns into which field values are loaded. For proper interpretation of the file, you must ensure that it was written with the correct character set. For example, if you write a data file with **mysqldump -T** or by issuing a `SELECT ... INTO OUTFILE` statement in **mysql**, be sure to use a `--default-character-set` option to write output in the character set to be used when the file is loaded with `LOAD DATA`.

> **Note**
>
> It is not possible to load data files that use the `ucs2`, `utf16`, `utf16le`, or `utf32` character set.

Input File Location

These rules determine the `LOAD DATA` input file location:

- If `LOCAL` is not specified, the file must be located on the server host. The server reads the file directly, locating it as follows:
  - If the file name is an absolute path name, the server uses it as given.
  - If the file name is a relative path name with leading components, the server looks for the file relative to its data directory.
  - If the file name has no leading components, the server looks for the file in the database directory of the default database.
- If `LOCAL` is specified, the file must be located on the client host. The client program reads the file, locating it as follows:
  - If the file name is an absolute path name, the client program uses it as given.

- If the file name is a relative path name, the client program looks for the file relative to its invocation directory.

When LOCAL is used, the client program reads the file and sends its contents to the server. The server creates a copy of the file in the directory where it stores temporary files. See Section B.3.3.5, “Where MySQL Stores Temporary Files”. Lack of sufficient space for the copy in this directory can cause the LOAD DATA LOCAL statement to fail.

The non-LOCAL rules mean that the server reads a file named as `./myfile.txt` relative to its data directory, whereas it reads a file named as `myfile.txt` from the database directory of the default database. For example, if the following LOAD DATA statement is executed while `db1` is the default database, the server reads the file `data.txt` from the database directory for `db1`, even though the statement explicitly loads the file into a table in the `db2` database:

```
LOAD DATA INFILE 'data.txt' INTO TABLE db2.my_table;
```

> **Note**
>
> The server also uses the non-LOCAL rules to locate `.sdi` files for the IMPORT TABLE statement.

Security Requirements

For a non-LOCAL load operation, the server reads a text file located on the server host, so these security requirements must be satisfied:

- You must have the FILE privilege. See Section 8.2.2, “Privileges Provided by MySQL”.
- The operation is subject to the secure_file_priv system variable setting:
  - If the variable value is a nonempty directory name, the file must be located in that directory.
  - If the variable value is empty (which is insecure), the file need only be readable by the server.

For a LOCAL load operation, the client program reads a text file located on the client host. Because the file contents are sent over the connection by the client to the server, using LOCAL is a bit slower than when the server accesses the file directly. On the other hand, you do not need the FILE privilege, and the file can be located in any directory the client program can access.

Duplicate-Key and Error Handling

The REPLACE and IGNORE modifiers control handling of new (input) rows that duplicate existing table rows on unique key values (PRIMARY KEY or UNIQUE index values):

- With REPLACE, new rows that have the same value as a unique key value in an existing row replace the existing row. See Section 15.2.12, “REPLACE Statement”.
- With IGNORE, new rows that duplicate an existing row on a unique key value are discarded. For more information, see The Effect of IGNORE on Statement Execution.

The LOCAL modifier has the same effect as IGNORE. This occurs because the server has no way to stop transmission of the file in the middle of the operation.

If none of REPLACE, IGNORE, or LOCAL is specified, an error occurs when a duplicate key value is found, and the rest of the text file is ignored.

In addition to affecting duplicate-key handling as just described, IGNORE and LOCAL also affect error handling:

- When neither IGNORE nor LOCAL is specified, data-interpretation errors terminate the operation.
- When IGNORE—or LOCAL without REPLACE—is specified, data interpretation errors become warnings and the load operation continues, even if the SQL mode is restrictive. For examples, see Column Value Assignment.

Index Handling

To ignore foreign key constraints during the load operation, execute a SET foreign_key_checks = 0 statement before executing LOAD DATA.

If you use LOAD DATA on an empty MyISAM table, all nonunique indexes are created in a separate batch (as for REPAIR TABLE). Normally, this makes LOAD DATA much faster when you have many indexes. In some extreme cases, you can create the indexes even faster by turning them off with ALTER TABLE ... DISABLE KEYS before loading the file into the table and re-creating the indexes with ALTER TABLE ... ENABLE KEYS after loading the file. See Section 10.2.5.1, “Optimizing INSERT Statements”.

Field and Line Handling

For both the LOAD DATA and SELECT ... INTO OUTFILE statements, the syntax of the FIELDS and LINES clauses is the same. Both clauses are optional, but FIELDS must precede LINES if both are specified.

If you specify a `FIELDS` clause, each of its subclauses (`TERMINATED BY`, `[OPTIONALLY] ENCLOSED BY`, and `ESCAPED BY`) is also optional, except that you must specify at least one of them. Arguments to these clauses are permitted to contain only ASCII characters.

If you specify no `FIELDS` or `LINES` clause, the defaults are the same as if you had written this:

```
FIELDS TERMINATED BY '\t' ENCLOSED BY '' ESCAPED BY '\\'
LINES TERMINATED BY '\n' STARTING BY ''
```

Backslash is the MySQL escape character within strings in SQL statements. Thus, to specify a literal backslash, you must specify two backslashes for the value to be interpreted as a single backslash. The escape sequences `'\t'` and `'\n'` specify tab and newline characters, respectively.

In other words, the defaults cause `LOAD DATA` to act as follows when reading input:

- Look for line boundaries at newlines.
- Do not skip any line prefix.
- Break lines into fields at tabs.
- Do not expect fields to be enclosed within any quoting characters.
- Interpret characters preceded by the escape character `\` as escape sequences. For example, `\t`, `\n`, and `\\` signify tab, newline, and backslash, respectively. See the discussion of `FIELDS ESCAPED BY` later for the full list of escape sequences.

Conversely, the defaults cause `SELECT ... INTO OUTFILE` to act as follows when writing output:

- Write tabs between fields.
- Do not enclose fields within any quoting characters.
- Use `\` to escape instances of tab, newline, or `\` that occur within field values.
- Write newlines at the ends of lines.

> **Note**
>
> For a text file generated on a Windows system, proper file reading might require `LINES TERMINATED BY '\r\n'` because Windows programs typically use two characters as a line terminator. Some programs, such as **WordPad**, might use `\r` as a

> line terminator when writing files. To read such files, use `LINES TERMINATED BY '\r'`.

If all the input lines have a common prefix that you want to ignore, you can use `LINES STARTING BY 'prefix_string'` to skip the prefix *and anything before it*. If a line does not include the prefix, the entire line is skipped. Suppose that you issue the following statement:

```
LOAD DATA INFILE '/tmp/test.txt' INTO TABLE test
  FIELDS TERMINATED BY ','  LINES STARTING BY 'xxx';
```

If the data file looks like this:

```
xxx"abc",1
something xxx"def",2
"ghi",3
```

The resulting rows are `("abc",1)` and `("def",2)`. The third row in the file is skipped because it does not contain the prefix.

The `IGNORE number LINES` clause can be used to ignore lines at the start of the file. For example, you can use `IGNORE 1 LINES` to skip an initial header line containing column names:

```
LOAD DATA INFILE '/tmp/test.txt' INTO TABLE test IGNORE 1 LINES;
```

When you use `SELECT ... INTO OUTFILE` in tandem with `LOAD DATA` to write data from a database into a file and then read the file back into the database later, the field- and line-handling options for both statements must match. Otherwise, `LOAD DATA` does not interpret the contents of the file properly. Suppose that you use `SELECT ... INTO OUTFILE` to write a file with fields delimited by commas:

```
SELECT * INTO OUTFILE 'data.txt'
  FIELDS TERMINATED BY ','
  FROM table2;
```

To read the comma-delimited file, the correct statement is:

```
LOAD DATA INFILE 'data.txt' INTO TABLE table2
  FIELDS TERMINATED BY ',';
```

If instead you tried to read the file with the statement shown following, it would not work because it instructs `LOAD DATA` to look for tabs between fields:

```
LOAD DATA INFILE 'data.txt' INTO TABLE table2
  FIELDS TERMINATED BY '\t';
```

The likely result is that each input line would be interpreted as a single field.

`LOAD DATA` can be used to read files obtained from external sources. For example, many programs can export data in comma-separated values (CSV) format, such that lines have fields separated by commas and enclosed within double quotation marks, with an initial line of column names. If the lines in such a file are terminated by carriage return/newline pairs, the statement shown here illustrates the field- and line-handling options you would use to load the file:

```
LOAD DATA INFILE 'data.txt' INTO TABLE tbl_name
  FIELDS TERMINATED BY ',' ENCLOSED BY '"'
  LINES TERMINATED BY '\r\n'
  IGNORE 1 LINES;
```

If the input values are not necessarily enclosed within quotation marks, use `OPTIONALLY` before the `ENCLOSED BY` option.

Any of the field- or line-handling options can specify an empty string (`''`). If not empty, the `FIELDS [OPTIONALLY] ENCLOSED BY` and `FIELDS ESCAPED BY` values must be a single character. The `FIELDS TERMINATED BY`, `LINES STARTING BY`, and `LINES TERMINATED BY` values can be more than one character. For example, to write lines that are terminated by carriage return/linefeed pairs, or to read a file containing such lines, specify a `LINES TERMINATED BY '\r\n'` clause.

To read a file containing jokes that are separated by lines consisting of `%%`, you can do this

```
CREATE TABLE jokes
  (a INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  joke TEXT NOT NULL);
LOAD DATA INFILE '/tmp/jokes.txt' INTO TABLE jokes
```

```
  FIELDS TERMINATED BY ''
  LINES TERMINATED BY '\n%%\n' (joke);
```

`FIELDS [OPTIONALLY] ENCLOSED BY` controls quoting of fields. For output (`SELECT ... INTO OUTFILE`), if you omit the word `OPTIONALLY`, all fields are enclosed by the `ENCLOSED BY` character. An example of such output (using a comma as the field delimiter) is shown here:

```
"1","a string","100.20"
"2","a string containing a , comma","102.20"
"3","a string containing a \" quote","102.20"
"4","a string containing a \", quote and comma","102.20"
```

If you specify `OPTIONALLY`, the `ENCLOSED BY` character is used only to enclose values from columns that have a string data type (such as `CHAR`, `BINARY`, `TEXT`, or `ENUM`):

```
1,"a string",100.20
2,"a string containing a , comma",102.20
3,"a string containing a \" quote",102.20
4,"a string containing a \", quote and comma",102.20
```

Occurrences of the `ENCLOSED BY` character within a field value are escaped by prefixing them with the `ESCAPED BY` character. Also, if you specify an empty `ESCAPED BY` value, it is possible to inadvertently generate output that cannot be read properly by `LOAD DATA`. For example, the preceding output just shown would appear as follows if the escape character is empty. Observe that the second field in the fourth line contains a comma following the quote, which (erroneously) appears to terminate the field:

```
1,"a string",100.20
2,"a string containing a , comma",102.20
3,"a string containing a " quote",102.20
4,"a string containing a ", quote and comma",102.20
```

For input, the `ENCLOSED BY` character, if present, is stripped from the ends of field values. (This is true regardless of whether `OPTIONALLY` is specified; `OPTIONALLY` has no effect on input interpretation.) Occurrences of the `ENCLOSED BY` character preceded by the `ESCAPED BY` character are interpreted as part of the current field value.

If the field begins with the `ENCLOSED BY` character, instances of that character are recognized as terminating a field value only if followed by the field or line `TERMINATED BY` sequence. To avoid ambiguity,

occurrences of the `ENCLOSED BY` character within a field value can be doubled and are interpreted as a single instance of the character. For example, if `ENCLOSED BY '"'` is specified, quotation marks are handled as shown here:

```
"The ""BIG"" boss"  -> The "BIG" boss
The "BIG" boss      -> The "BIG" boss
The ""BIG"" boss    -> The ""BIG"" boss
```

`FIELDS ESCAPED BY` controls how to read or write special characters:

- For input, if the `FIELDS ESCAPED BY` character is not empty, occurrences of that character are stripped and the following character is taken literally as part of a field value. Some two-character sequences that are exceptions, where the first character is the escape character. These sequences are shown in the following table (using `\` for the escape character). The rules for `NULL` handling are described later in this section.

Character	Escape Sequence
`\0`	An ASCII NUL (`X'00'`) character
`\b`	A backspace character
`\n`	A newline (linefeed) character
`\r`	A carriage return character
`\t`	A tab character.
`\Z`	ASCII 26 (Control+Z)
`\N`	NULL

  For more information about `\`-escape syntax, see Section 11.1.1, “String Literals”.

  If the `FIELDS ESCAPED BY` character is empty, escape-sequence interpretation does not occur.

- For output, if the `FIELDS ESCAPED BY` character is not empty, it is used to prefix the following characters on output:

  - The `FIELDS ESCAPED BY` character.
  - The `FIELDS [OPTIONALLY] ENCLOSED BY` character.
  - The first character of the `FIELDS TERMINATED BY` and `LINES TERMINATED BY` values, if the `ENCLOSED BY` character is empty or unspecified.

  - ASCII `0` (what is actually written following the escape character is ASCII `0`, not a zero-valued byte).

  If the `FIELDS ESCAPED BY` character is empty, no characters are escaped and `NULL` is output as `NULL`, not `\N`. It is probably not a good idea to specify an empty escape character, particularly if field values in your data contain any of the characters in the list just given.

In certain cases, field- and line-handling options interact:

- If `LINES TERMINATED BY` is an empty string and `FIELDS TERMINATED BY` is nonempty, lines are also terminated with `FIELDS TERMINATED BY`.

- If the `FIELDS TERMINATED BY` and `FIELDS ENCLOSED BY` values are both empty (`''`), a fixed-row (nondelimited) format is used. With fixed-row format, no delimiters are used between fields (but you can still have a line terminator). Instead, column values are read and written using a field width wide enough to hold all values in the field. For `TINYINT`, `SMALLINT`, `MEDIUMINT`, `INT`, and `BIGINT`, the field widths are 4, 6, 8, 11, and 20, respectively, no matter what the declared display width is.

  `LINES TERMINATED BY` is still used to separate lines. If a line does not contain all fields, the rest of the columns are set to their default values. If you do not have a line terminator, you should set this to `''`. In this case, the text file must contain all fields for each row.

  Fixed-row format also affects handling of `NULL` values, as described later.

  > **Note**
  >
  > Fixed-size format does not work if you are using a multibyte character set.

Handling of `NULL` values varies according to the `FIELDS` and `LINES` options in use:

- For the default `FIELDS` and `LINES` values, `NULL` is written as a field value of `\N` for output, and a field value of `\N` is read as `NULL` for input (assuming that the `ESCAPED BY` character is `\`).

- If `FIELDS ENCLOSED BY` is not empty, a field containing the literal word `NULL` as its value is read as a `NULL` value. This differs from the word `NULL` enclosed within `FIELDS ENCLOSED BY` characters, which is read as the string `'NULL'`.

- If `FIELDS ESCAPED BY` is empty, `NULL` is written as the word `NULL`.

- With fixed-row format (which is used when `FIELDS TERMINATED BY` and `FIELDS ENCLOSED BY` are both empty), `NULL` is written as an empty string. This causes both `NULL` values and empty strings in the table to be indistinguishable when written to the file because both are written as empty strings. If

you need to be able to tell the two apart when reading the file back in, you should not use fixed-row format.

An attempt to load `NULL` into a `NOT NULL` column produces either a warning or an error according to the rules described in Column Value Assignment.

Some cases are not supported by `LOAD DATA`:

- Fixed-size rows (`FIELDS TERMINATED BY` and `FIELDS ENCLOSED BY` both empty) and `BLOB` or `TEXT` columns.
- If you specify one separator that is the same as or a prefix of another, `LOAD DATA` cannot interpret the input properly. For example, the following `FIELDS` clause would cause problems:

```
FIELDS TERMINATED BY '"' ENCLOSED BY '"'
```

- If `FIELDS ESCAPED BY` is empty, a field value that contains an occurrence of `FIELDS ENCLOSED BY` or `LINES TERMINATED BY` followed by the `FIELDS TERMINATED BY` value causes `LOAD DATA` to stop reading a field or line too early. This happens because `LOAD DATA` cannot properly determine where the field or line value ends.

Column List Specification

The following example loads all columns of the `persondata` table:

```
LOAD DATA INFILE 'persondata.txt' INTO TABLE persondata;
```

By default, when no column list is provided at the end of the `LOAD DATA` statement, input lines are expected to contain a field for each table column. If you want to load only some of a table's columns, specify a column list:

```
LOAD DATA INFILE 'persondata.txt' INTO TABLE persondata
(col_name_or_user_var [, col_name_or_user_var] ...);
```

You must also specify a column list if the order of the fields in the input file differs from the order of the columns in the table. Otherwise, MySQL cannot tell how to match input fields with table columns.

Input Preprocessing

Each instance of ***col_name_or_user_var*** in LOAD DATA syntax is either a column name or a user variable. With user variables, the SET clause enables you to perform preprocessing transformations on their values before assigning the result to columns.

User variables in the SET clause can be used in several ways. The following example uses the first input column directly for the value of `t1.column1`, and assigns the second input column to a user variable that is subjected to a division operation before being used for the value of `t1.column2`:

```
LOAD DATA INFILE 'file.txt'
  INTO TABLE t1
  (column1, @var1)
  SET column2 = @var1/100;
```

The SET clause can be used to supply values not derived from the input file. The following statement sets `column3` to the current date and time:

```
LOAD DATA INFILE 'file.txt'
  INTO TABLE t1
  (column1, column2)
  SET column3 = CURRENT_TIMESTAMP;
```

You can also discard an input value by assigning it to a user variable and not assigning the variable to any table column:

```
LOAD DATA INFILE 'file.txt'
  INTO TABLE t1
  (column1, @dummy, column2, @dummy, column3);
```

Use of the column/variable list and SET clause is subject to the following restrictions:

- Assignments in the SET clause should have only column names on the left hand side of assignment operators.
- You can use subqueries in the right hand side of SET assignments. A subquery that returns a value to be assigned to a column may be a scalar subquery only. Also, you cannot use a subquery to select from the table that is being loaded.

- Lines ignored by an `IGNORE *number* LINES` clause are not processed for the column/variable list or `SET` clause.
- User variables cannot be used when loading data with fixed-row format because user variables do not have a display width.

Column Value Assignment

To process an input line, `LOAD DATA` splits it into fields and uses the values according to the column/variable list and the `SET` clause, if they are present. Then the resulting row is inserted into the table. If there are `BEFORE INSERT` or `AFTER INSERT` triggers for the table, they are activated before or after inserting the row, respectively.

Interpretation of field values and assignment to table columns depends on these factors:

- The SQL mode (the value of the `sql_mode` system variable). The mode can be nonrestrictive, or restrictive in various ways. For example, strict SQL mode can be enabled, or the mode can include values such as `NO_ZERO_DATE` or `NO_ZERO_IN_DATE`.
- Presence or absence of the `IGNORE` and `LOCAL` modifiers.

Those factors combine to produce restrictive or nonrestrictive data interpretation by `LOAD DATA`:

- Data interpretation is restrictive if the SQL mode is restrictive and neither the `IGNORE` nor the `LOCAL` modifier is specified. Errors terminate the load operation.
- Data interpretation is nonrestrictive if the SQL mode is nonrestrictive or the `IGNORE` or `LOCAL` modifier is specified. (In particular, either modifier if specified *overrides* a restrictive SQL mode when the `REPLACE` modifier is omitted.) Errors become warnings and the load operation continues.

Restrictive data interpretation uses these rules:

- Too many or too few fields results an error.
- Assigning `NULL` (that is, `\N`) to a non-`NULL` column results in an error.
- A value that is out of range for the column data type results in an error.
- Invalid values produce errors. For example, a value such as `'x'` for a numeric column results in an error, not conversion to 0.

By contrast, nonrestrictive data interpretation uses these rules:

- If an input line has too many fields, the extra fields are ignored and the number of warnings is incremented.
- If an input line has too few fields, the columns for which input fields are missing are assigned their default values. Default value assignment is described in Section 13.6, “Data Type Default Values”.
- Assigning `NULL` (that is, `\N`) to a non-`NULL` column results in assignment of the implicit default value for the column data type. Implicit default values are described in Section 13.6, “Data Type Default Values”.
- Invalid values produce warnings rather than errors, and are converted to the “closest” valid value for the column data type. Examples:
  - A value such as `'x'` for a numeric column results in conversion to 0.
  - An out-of-range numeric or temporal value is clipped to the closest endpoint of the range for the column data type.
  - An invalid value for a `DATETIME`, `DATE`, or `TIME` column is inserted as the implicit default value, regardless of the SQL mode `NO_ZERO_DATE` setting. The implicit default is the appropriate “zero” value for the type (`'0000-00-00 00:00:00'`, `'0000-00-00'`, or `'00:00:00'`). See Section 13.2, “Date and Time Data Types”.
- `LOAD DATA` interprets an empty field value differently from a missing field:
  - For string types, the column is set to the empty string.
  - For numeric types, the column is set to `0`.
  - For date and time types, the column is set to the appropriate “zero” value for the type. See Section 13.2, “Date and Time Data Types”.

  These are the same values that result if you assign an empty string explicitly to a string, numeric, or date or time type explicitly in an `INSERT` or `UPDATE` statement.

`TIMESTAMP` columns are set to the current date and time only if there is a `NULL` value for the column (that is, `\N`) and the column is not declared to permit `NULL` values, or if the `TIMESTAMP` column default value is the current timestamp and it is omitted from the field list when a field list is specified.

`LOAD DATA` regards all input as strings, so you cannot use numeric values for `ENUM` or `SET` columns the way you can with `INSERT` statements. All `ENUM` and `SET` values must be specified as strings.

`BIT` values cannot be loaded directly using binary notation (for example, `b'011010'`). To work around this, use the `SET` clause to strip off the leading `b'` and trailing `'` and perform a base-2 to base-10 conversion so that MySQL loads the values into the `BIT` column properly:

```
$> cat /tmp/bit_test.txt
b'10'
b'1111111'
$> mysql test
mysql> LOAD DATA INFILE '/tmp/bit_test.txt'
       INTO TABLE bit_test (@var1)
       SET b = CAST(CONV(MID(@var1, 3, LENGTH(@var1)-3), 2, 10) AS UNSIGNED);
Query OK, 2 rows affected (0.00 sec)
Records: 2  Deleted: 0  Skipped: 0  Warnings: 0

mysql> SELECT BIN(b+0) FROM bit_test;
+----------+
| BIN(b+0) |
+----------+
| 10       |
| 1111111  |
+----------+
2 rows in set (0.00 sec)
```

For `BIT` values in `0b` binary notation (for example, `0b011010`), use this `SET` clause instead to strip off the leading `0b`:

```
SET b = CAST(CONV(MID(@var1, 3, LENGTH(@var1)-2), 2, 10) AS UNSIGNED)
```

Partitioned Table Support

`LOAD DATA` supports explicit partition selection using the `PARTITION` clause with a list of one or more comma-separated names of partitions, subpartitions, or both. When this clause is used, if any rows from the file cannot be inserted into any of the partitions or subpartitions named in the list, the statement fails with the error `Found a row not matching the given partition set`. For more information and examples, see Section 26.5, “Partition Selection”.

Concurrency Considerations

With the `LOW_PRIORITY` modifier, execution of the `LOAD DATA` statement is delayed until no other clients are reading from the table. This affects only storage engines that use only table-level locking (such as `MyISAM`, `MEMORY`, and `MERGE`).

With the `CONCURRENT` modifier and a `MyISAM` table that satisfies the condition for concurrent inserts (that is, it contains no free blocks in the middle), other threads can retrieve data from the table while `LOAD DATA` is executing. This modifier affects the performance of `LOAD DATA` a bit, even if no other thread is using the table at the same time.

Statement Result Information

When the `LOAD DATA` statement finishes, it returns an information string in the following format:

```
Records: 1  Deleted: 0  Skipped: 0  Warnings: 0
```

Warnings occur under the same circumstances as when values are inserted using the `INSERT` statement (see Section 15.2.7, “INSERT Statement”), except that `LOAD DATA` also generates warnings when there are too few or too many fields in the input row.

You can use `SHOW WARNINGS` to get a list of the first `max_error_count` warnings as information about what went wrong. See Section 15.7.7.42, “SHOW WARNINGS Statement”.

If you are using the C API, you can get information about the statement by calling the `mysql_info()` function. See mysql_info().

Replication Considerations

`LOAD DATA` is considered unsafe for statement-based replication. If you use `LOAD DATA` with `binlog_format=STATEMENT`, each replica on which the changes are to be applied creates a temporary file containing the data. This temporary file is not encrypted, even if binary log encryption is active on the source, If encryption is required, use row-based or mixed binary logging format instead, for which replicas do not create the temporary file. For more information on the interaction between `LOAD DATA` and replication, see Section 19.5.1.19, “Replication and LOAD DATA”.

Miscellaneous Topics

On Unix, if you need `LOAD DATA` to read from a pipe, you can use the following technique (the example loads a listing of the `/` directory into the table `db1.t1`):

```
mkfifo /mysql/data/db1/ls.dat
chmod 666 /mysql/data/db1/ls.dat
find / -ls > /mysql/data/db1/ls.dat &
mysql -e "LOAD DATA INFILE 'ls.dat' INTO TABLE t1" db1
```

Here you must run the command that generates the data to be loaded and the **mysql** commands either on separate terminals, or run the data generation process in the background (as shown in the preceding example). If you do not do this, the pipe blocks until data is read by the **mysql** process.